



CS230

Section 9 (Week 9)

Writing a Deep Learning Paper (Featuring Previous Projects!)

General Ideas

When writing your paper, always remember your audience. In particular,

- Your audience is already well-versed in deep learning. Hence, there is no need to describe in detail common deep learning techniques. For example, you can refer to methods such as convolutions, pooling, dropout, batch normalization, recurrent neural networks or L2 regularization without explaining their meaning. If you are using well-known data augmentation methods such as image rotation and translation, you can mention it without showing the augmented images.
- Your audience is busy. Hence, be as short and concise as possible and focus on the key insights of your paper. If the key insight of your paper is the loss function, write out the loss function and elaborate on it.

If the key insight of your paper is the model architecture, draw it out and explain the key differences with prior work. For all other details that are common in deep learning, it's best to state it concisely.

- Your audience may want to reimplement your paper. Hence, it's important to give details which will help them to reimplement it. Once again, there's no need to go overboard with details. Think about what will help you re-implement your own paper, and what were the main difficulties you faced when implementing your project were: elaborate more on those points.

The abstract, introduction and conclusion are essential parts of a paper. When writing these parts, it's important to describe the *context* of your paper, the *motivation* behind your work and the *key insights* drawn from your work. It is also good to mention some results to entice your audience to read more. Refer to the section below on how to write an abstract for more details.

It is also important to provide explanations behind your decisions. Instead of stating that you tuned the number of frozen layers in your network, go beyond that and explain why you believe that a certain hyperparameter is the most important to tune in your network. Doing so also helps your audience fine-tune your network for their own dataset.

How to write an abstract?

Abstract

As techniques for creating photo realistic imagery evolve, it becomes increasingly difficult to differentiate between real photos and computer generated (CG) images. We utilize custom high pass filters and a CNN framework to take CG and photo inputs and classify them as CG or real. Building on past projects for photo vs. CG differentiation, we achieved a 99.7% accuracy on 100x100 resolution image blocks, outperforming current state of the art published results.

Realistic Image Synthesis and Classification - Chris Fontas, Wendy Li, Emanuel Mendiola, Fall 2018 ([source](#))

In general, the abstract (and the introduction) should address the following questions (from Professor Emma Brunskill):

- What is the problem to be addressed?
- Why is the problem important and why is it hard?
- What are the key limitations of prior work (with associated references)?
- What are the key insights of this paper?
- What are the key results in this paper?

Why is this good?

- Concise and clear
- Addresses most of the above questions

How to describe prior work?

2. Related Work

Face Alignment: Deep face alignment networks are generally classified into coordinate regression models [11, 13, 27, 31], where a direct mapping is learned between the image and the landmark coordinates, and heatmap regression models [2, 4, 32], where prediction heatmaps are learned for each landmark. Heatmap-based architectures are generally derived from stacked hourglass [2, 4, 10, 21] or convolutional pose machine [29] architectures used for human body pose estimation. Pixel coordinates can be obtained from the heatmaps by applying the argmax operation; however, [5, 26] use soft-argmax to achieve end-to-end differentiability.

Optical Flow: In this project we focus on deep optical flow networks. End-to-end methods for learning optical flow using deep networks were first proposed by [6] and later refined in [9]. Other methods include DeepFlow [30], etc. use deep networks to detect correspondences. These methods are generally evaluated on the Middlebury dataset [1], the KITTI dataset [8], and the MPI Sintel dataset [3].

Using deep networks such as VGG-16 [25] for losses has been shown to be effective for training other deep networks for tasks such as style transfer and super-resolution [12]. Furthermore, deep networks have been used in energies for traditional optimization problems for style transfer [7], texture synthesis [24], and image generation [19, 28].

Why is this good?

- Classifies relevant prior work in broad categories. For each category, papers are grouped by their more specific topics.
- Related work is not just mentioned: its relation to this work is explicitly shown.
- The writing is clear and succinct. As a rule of thumb, when writing a paper, remove any words that aren't bringing meaningful information. For instance, "our convolutional neural network performs object detection by outputting all-sized bounding boxes" could be written as "our object detector outputs bounding boxes" if there's no need for the reader to know that the object detector is a CNN and outputs bounding boxes of all-size.
- References are mentioned throughout the paragraph to help a curious reader delve deeper into each discussed topic.

How can this be improved?

- Elaborate more on limitations/benefits of prior work. For instance, briefly mention what previous optical flow networks tend to succeed or fail on.
- Elaborate more on which prior work inspired your work, if any.

How to describe or visualize your dataset?

3 Dataset and Features

Our dataset is from Kaggle Human Protein Atlas Image Classification website [1] as part of the Human Protein Atlas program [2]). We have 31072 samples, each sample consists of 512 x 512 images from four (RGBY) color filters. The green channel shows the location of the protein obtained by immunofluorescence microscopy after staining using targeted antibodies. The other 3 channels (blue, green and yellow) shows the different regions of the cell. Each samples can have one or more labels from 28 different cell regions corresponding to sub-cellular structure of the protein to be localized. Our dataset has class imbalance that makes the classification problem challenging. For e.g. Nucleoplasm (Class0) is present in 12885 samples, whereas Rods and Rings (Class27) is present in only 11 samples. Lysosomes, Microtubule ends and Mitotic spindle (Classes10, 15 and 17) are not present as unique labels but along with other classes only.

Human Protein Atlas Image Classification - Amita Patil, Rudra Bandhu, Fall 2018 ([source](#))

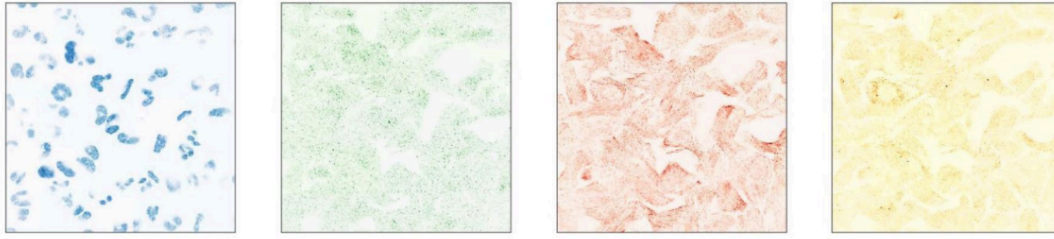


Figure 1: Representative sample image with multiple labels: Nucleoplasm and Microtubule organizing center.

3.1 Data Preprocessing

5% of both unique and multilabel samples along with at least 5 samples from each class were held out for validation. Validation set includes 1576 samples. All images were downsampled from 512 x 512 to 224 x 224 so we could use pretrained networks such as ResNet18 and DenseNet161 for finetuning. We performed two levels of data augmentation on training samples

- 1) 3 rotations: 90°, 180°, 270° and 2 mirrorings: horizontal and vertical flips
- 2) 7 rotations: 45° through 315° in 45° steps and the 2 mirrorings.

Human Protein Atlas Image Classification - Amita Patil, Rudra Bandhu, Fall 2018 ([source](#))

Why is this good?

- The authors analyzed their dataset and reported key figures such as dataset size and format. They also displayed sample images from the dataset to help the reader grasp the problem they are solving.
- They also discussed key challenges related to their dataset such as the class imbalance and low-data regime.
- They mentioned using data augmentation and preprocessing to alleviate the data-related challenges.
- You may also include dataset collection/scraping if you've done so.
- Other ways to visualize data: word cloud, histograms, etc.

What should be improved?

- The authors haven't explicitly formulated the split between training, validation and test sets. Be sure to include this, either in your dataset section or in your methods section!

How to describe a deep learning architecture?

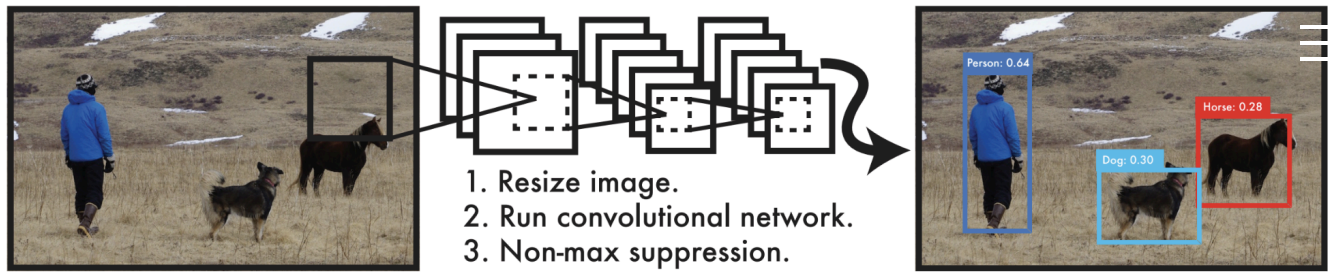
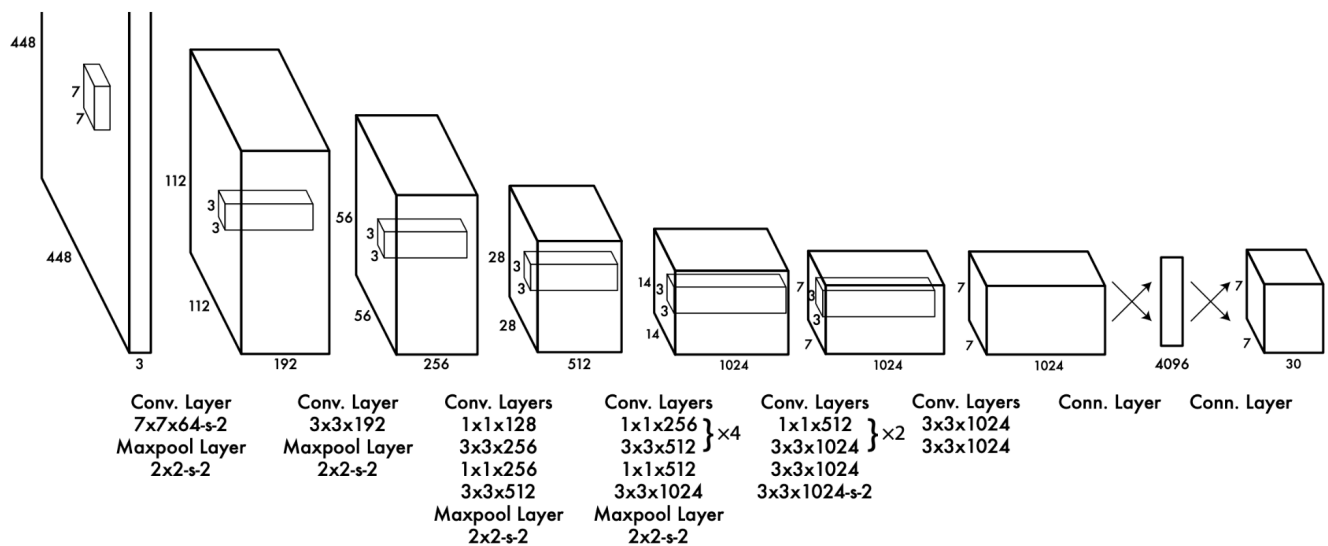


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

You Only Look Once: Unified, Real-Time Object Detection - Redmon et. al, 2015 ([source](#))



You Only Look Once: Unified, Real-Time Object Detection - Redmon et. al, 2015 ([source](#))

$$\begin{aligned}
& \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}$$

You Only Look Once: Unified, Real-Time Object Detection - Redmon et. al, 2015 ([source](#))

Why is this good?

- Model architecture is drawn clearly. Convolutions and maxpool are shown with their hyperparameters.
- Dimensions of each layer are also shown clearly for ease of re-implementation.
- Key insight of dividing image into grids and using each grid to do bounding box regression is clear from the first image. (Always remember what you want the audience to take away from your paper. It's not always just about drawing the model architecture. For example, if your key insight is a certain type of data augmentation for example, use a diagram to illustrate it and compare it with other types of data augmentation).
- Loss function is another key insight in this paper since it's about repurposing regressors for object detections. Hence, describing the loss function is importance in this case. But it's not always important. If you're using a common loss function such as cross-entropy loss, there's no need to write down the formula.

Other interesting points:

- Loss function: Why sqrt the width and height before taking L2 distance? (think ratios. If the actual width is 10 and prediction is 11 vs. if actual width is 1 and prediction is 2).

- Repurposing regressors as object detectors (vs. RCNN which uses classifiers as object detectors).

How to describe a deep learning architecture (for GANs)

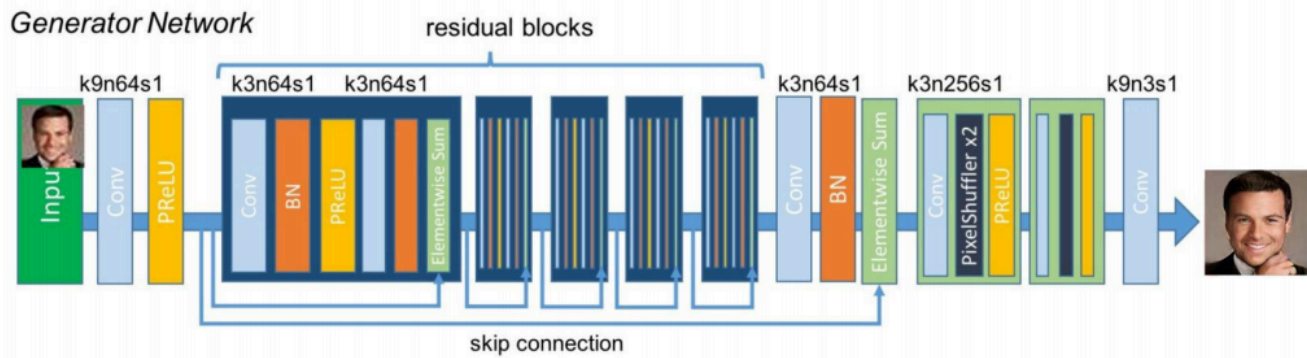


Figure 2: SRResnet model and Generator Network.

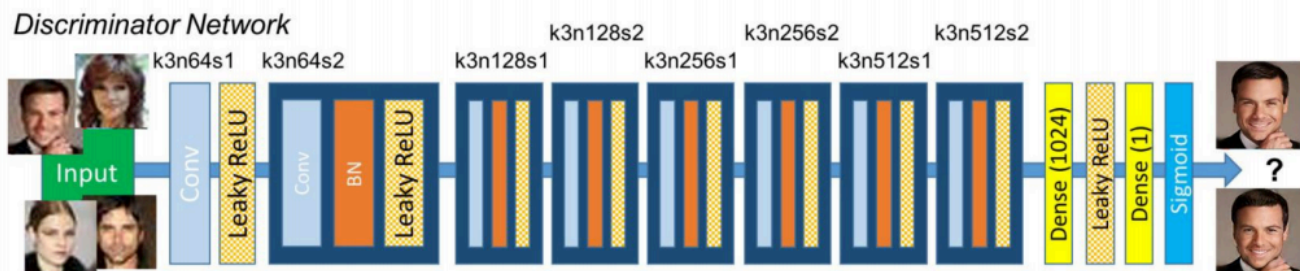


Figure 3: Discriminator Network.

Human Portrait Super Resolution Using GANs - Yujie Shu, Fall 2018 ([source](#))

Why is this good?

- Clear description of model architecture of both generator and discriminator for ease of reimplementation.

How can this be improved?

- Also describe the output size at each layer for ease of re-implementation.

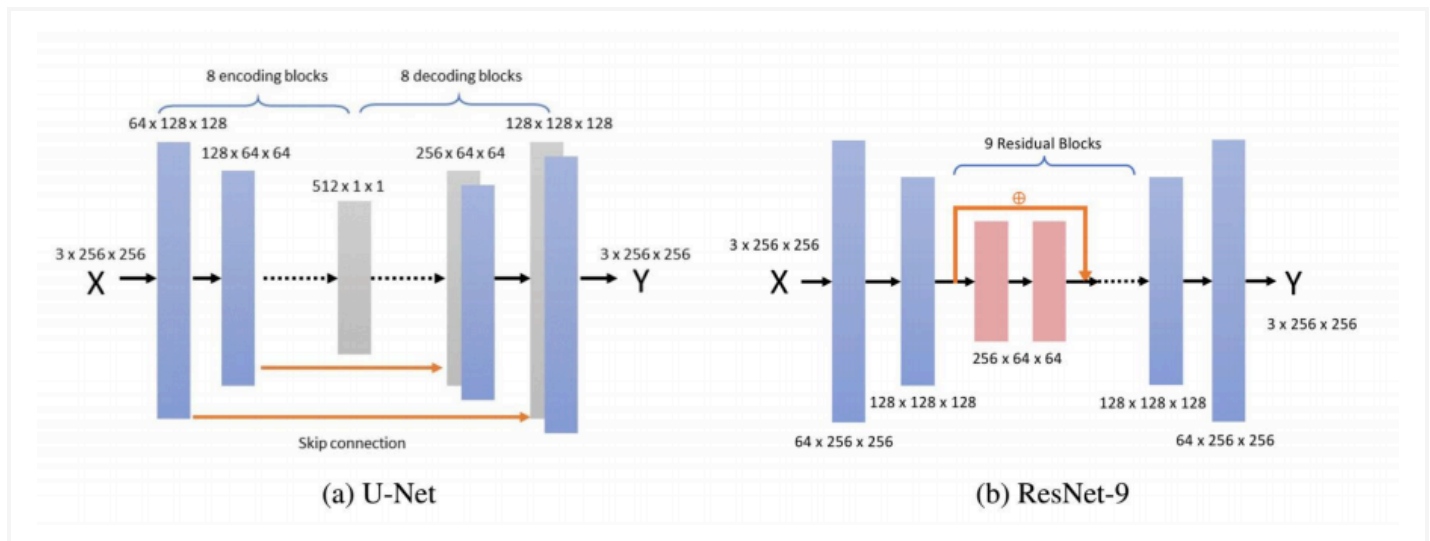


Image-to-Image Translation with Conditional-GAN - Jason Hu, Weiye Yu, Zhouchangwan Yu, Spring 2018 ([source](#))

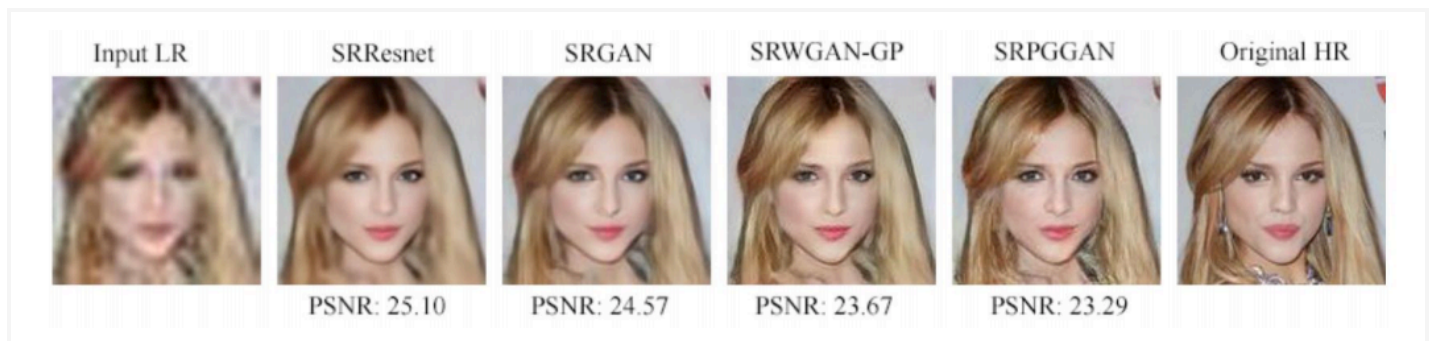
Why is this good?

- Quickly see the difference between the two types of architecture that are being compared: U-Net and ResNet.

How can this be improved?

- Describe the actual conv params and maxpool params used. (Refer to YOLOv1)

GAN Qualitative Evaluation



Human Portrait Super Resolution Using GANs - Yujie Shu, Fall 2018 ([source](#))

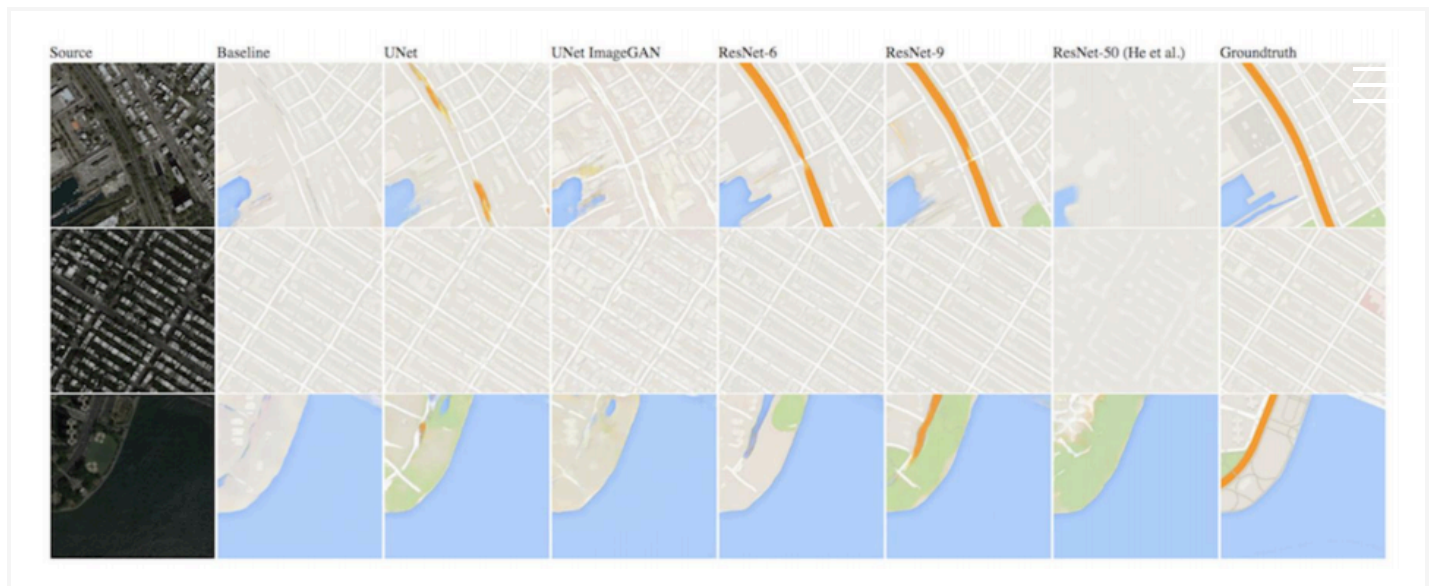
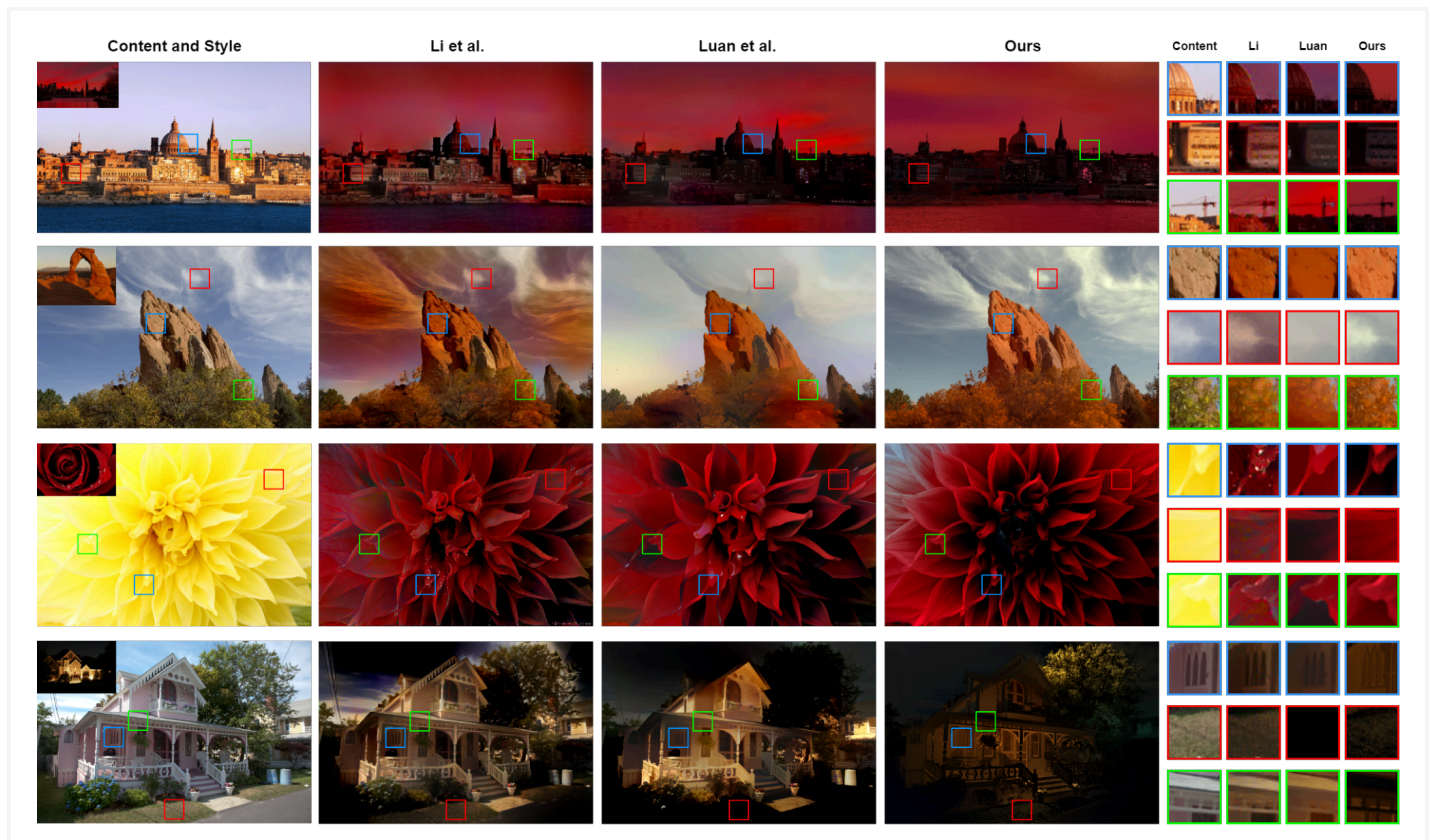


Image-to-Image Translation with Conditional-GAN - Jason Hu, Weiye Yu, Zhouchangwan Yu, Spring 2018 ([source](#))



Photorealistic Neural Style Transfer: Generating Realistic Images without GANs - Richard R. Yang, Fall 2018 ([source](#))

Why is this good?

- Quick comparison between different approaches
- Quantitative figures are also given in the top figure since the images themselves are hard to differentiate.
- See the CycleGAN paper for more nice figures.

How can this be improved?

- For more convincing evaluation, don't curate (hand pick and choose) your generated images, and tell your user your figures are uncurated. See <https://arxiv.org/pdf/1812.04948.pdf> for great examples.

GAN Quantitative Evaluation

- Classifying using GANs as Feature Extractor
- Inception Score
- Frechet Inception Distance
- For a great overview, see [Ali Borji's paper summarizing GAN evaluation measures.](#)

How to describe hyperparameter tuning steps?

A general approach could be:

Step 1: Describe which hyperparameters you tuned.

Step 2: Explain why you chose to tune these hyperparameters.

Step 3: Elaborate on how you tuned these hyperparameters. (e.g. Did you use a dev set or train-dev set or leave-one-out cross validation? Was there a variance/bias problem? What range of hyperparameters did you use? Any other fancy approaches such as Bayesian Optimization?)

Step 4: State the results of tuning these hyperparameters.

Step 5: Conclude by saying which hyperparameters you tuned were the most sensitive to the performance of your model.

The key here is in the explanation. Instead of blindly tuning all your hyperparameters, it is better to focus on a few hyperparameters that you believe is important. Doing so also helps the reader reimplement your paper since they understand which hyperparameters they should focus on tuning.

How to present classification results and handle unbalanced datasets?

	Network	Densenet73	Densenet161	Densenet161	Densenet161	Densenet161 + drop rate in dense layers = 0.3	Densenet161	Resnet1c
	Data Augmentation	None	None	None	Level1	Level1	Level2	Level1
Metrics (Macro)	Loss Weight Type	1	2	3	3	3	3	3
Accuracy	Train	0.57	0.24	0.92	0.97	0.84	0.98	0.94
	Dev	0.38	0.19	0.43	0.49	0.46	0.56	0.47
Precision	Train	0.82	0.58	0.98	0.99	0.975	0.996	0.99
	Dev	0.62	0.53	0.68	0.78	0.73	0.83	0.76
Recall	Train	0.57	0.23	0.97	0.99	0.96	0.995	0.99
	Dev	0.38	0.5	0.62	0.7	0.73	0.73	0.69
F1 score	Train	0.65	0.31	0.98	0.99	0.967	0.995	0.99
	Dev	0.43	0.48	0.64	0.73	0.7	0.76	0.71

Table 3: Performance summary of our models.

Human Protein Atlas Image Classification - Amita Patil, Rudra Bandhu, Fall 2018 ([source](#))

Why is this good?

- Presents all metrics clearly in the table with the most relevant hyperparameters and architectures.
- Highlights best results.
- Uses F1 score, Precision and Recall to evaluate since dataset is unbalanced.

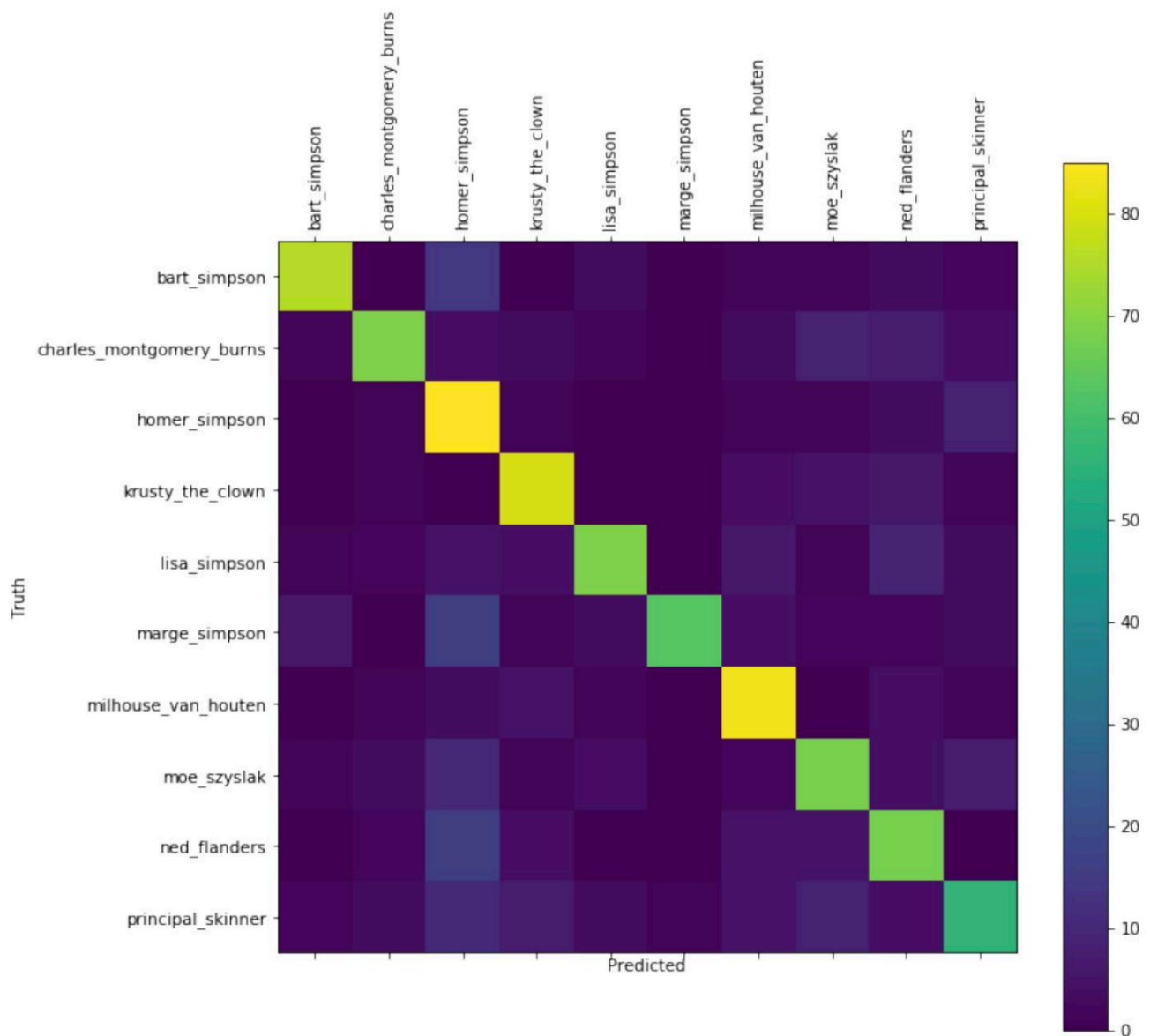


Figure 3. Confusion matrix of the testing result

Transfer Learning-based CNN Classification for Simpsons Characters - Yueheng Li, Winter 2019 [source](#)

Deep Architectural Style Classification

	Count	Baseline CNN			Final Architecture		
		F1	Precision	Recall	F1	Precision	Recall
Achaemenid	69	61.5	66.7	57.1	72.7	100.0	57.1
American Foursquare	59	25.0	50.0	16.7	28.6	100.0	16.7
American Craftsman	195	52.9	64.3	45.0	82.1	84.2	80.0
Ancient Egyptian	256	<u>98.0</u>	100.0	96.2	<u>98.0</u>	100.0	96.1
Art Deco	366	47.1	41.7	54.1	73.1	66.7	81.1
Art Nouveau	450	60.5	65.0	56.5	76.4	69.6	84.8
Baroque	239	57.8	61.9	54.2	73.1	88.2	62.5
Bauhaus	92	28.6	50.0	20.0	52.6	55.6	50.0
Beaux-Arts	191	50.0	56.3	45.0	55.6	62.5	50.0
Byzantine	111	20.0	25.0	16.7	81.8	90	75.0
Chicago School	153	51.4	47.4	56.3	69.2	90	56.3
Colonial	177	37.2	32.0	44.4	77.4	92.3	66.7
Deconstructivism	213	50.0	41.2	63.6	83.7	85.7	81.8
Edwardian	79	43.5	33.3	62.5	71.4	83.3	62.5
Georgian	154	47.1	44.4	50.0	66.7	64.7	68.8
Gothic	109	55.6	71.4	45.5	73.7	87.5	63.6
Greek Revival	327	<u>65.7</u>	64.7	66.7	80.0	71.4	90.9
International	207	53.7	55.0	52.4	53.3	88.9	38.1
Novelty	212	51.3	58.8	45.5	<u>89.4</u>	84.0	95.4
Palladian	113	43.5	45.5	41.7	58.3	58.3	58.3
Postmodern	163	44.4	60.0	35.3	57.1	48.0	70.6
Queen Anne	425	64.8	53.8	81.4	81.8	80.0	83.7
Romanesque	107	50.0	55.6	45.5	70.0	77.8	63.6
Russian Revival	165	48.6	45.0	52.9	84.2	76.2	94.1
Tudor Revival	162	29.6	40.0	23.5	62.2	50.0	82.4
No Architecture	212	<u>87.0</u>	100.0	76.9	<u>98.0</u>	100.0	96.1
Overall	5,053	55.1	57.4	55.4	75.3	78.3	75.7

Table 1. Results. Per-class counts and result metrics for our baseline and final model – we present standard F1, Recall, and Precision metrics. The best and worst few F1 scores for each model are underlined and bolded, respectively. Globally, Xu et al. (2014) reported 46.21% accuracy on a test set of 1,716 images across the 25 original architecture styles. Our baseline CNN's global accuracy was 55.4%, and our best approach's accuracy was 75.7%.

Deep Architectural Style Classification - Paavani Dua, Alan Flores-Lopez, Alex Wade, Winter 2019 ([source](#))

Why is this good?

- We can see how the classifier does on each individual class, as well as holistically. This works with any dataset balance!
- Having counts of number of examples of each class is great. Highlighting the best model for each row helps immensely with readability.

How to cite other publications and projects?

References

- [1] M. B. Blaschko and C. H. Lampert. Learning to localize objects with structured output regression. In *Computer Vision–ECCV 2008*, pages 2–15. Springer, 2008. [4](#)
- [2] L. Bourdev and J. Malik. Poselets: Body part detectors trained using 3d human pose annotations. In *International Conference on Computer Vision (ICCV)*, 2009. [8](#)
- [3] H. Cai, Q. Wu, T. Corradi, and P. Hall. The cross-depiction problem: Computer vision algorithms for recognising objects in artwork and in photographs. *arXiv preprint arXiv:1505.00110*, 2015. [7](#)
- [4] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on*, volume 1, pages 886–893. IEEE, 2005. [4](#), [8](#)
- [5] T. Dean, M. Ruzon, M. Segal, J. Shlens, S. Vijayanarasimhan, J. Yagnik, et al. Fast, accurate detection of 100,000 object classes on a single machine. In *Computer Vision and Pattern Recognition (CVPR), 2013 IEEE Conference on*, pages 1814–1821. IEEE, 2013. [5](#)
- [16] B. Hariharan, P. Arbeláez, R. Girshick, and J. Malik. Simultaneous detection and segmentation. In *Computer Vision–ECCV 2014*, pages 297–312. Springer, 2014. [7](#)
- [17] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *arXiv preprint arXiv:1406.4729*, 2014. [5](#)
- [18] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012. [4](#)
- [19] D. Hoiem, Y. Chodpathumwan, and Q. Dai. Diagnosing error in object detectors. In *Computer Vision–ECCV 2012*, pages 340–353. Springer, 2012. [6](#)
- [20] K. Lenc and A. Vedaldi. R-cnn minus r. *arXiv preprint arXiv:1506.06981*, 2015. [6](#)
- [21] R. Lienhart and J. Maydt. An extended set of haar-like features for rapid object detection. In *Image Processing. 2002. Proceedings. 2002 International Conference on*, volume 1, pages I–900. IEEE, 2002. [4](#)
- [22] M. Lin, Q. Chen, and S. Yan. Network in network. *CoRR*, abs/1312.4400, 2013. [2](#)

You Only Look Once: Unified, Real-Time Object Detection - Redmon et. al, 2015 ([source](#))

Why is this good:

- All authors are listed, and all citations are in a standard format.
- Conference/venue listed and year
- Can do this automatically using BibTeX or Word, with in-text citations linking to the referenced citation.

Copyright © 2025. - Stanford University

All rights reserved.